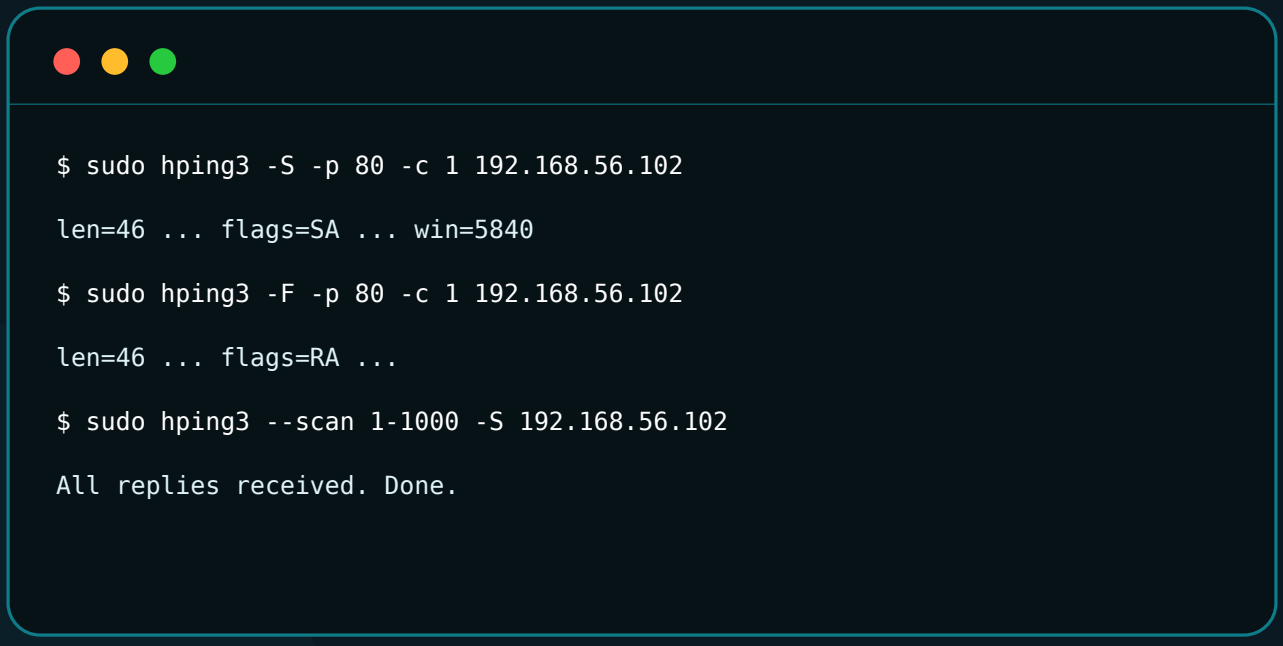
PRAXISHANDBUCH

hping3

Das Praxishandbuch

Pakete gezielt erzeugen, analysieren und testen – vom Portscan bis zur Firewall-Prüfung



JEDES FELD VERSTEHEN. JEDES PAKET KONTROLLIEREN. NUR IM EIGENEN NETZ.

Mit Übungen, Spickzettel, Glossar und Index · Ausgabe Oktober 2026

hping3
Das Praxishandbuch

Pakete gezielt erzeugen, analysieren und testen – vom Portscan bis zur Firewall-Prüfung

Vom TCP/IP-Fundament über Portscanning, Firewall-Tests und Idle Scan bis zu Scripting.
Mit geführten Übungen in einem eigenen, isolierten Testnetz, Spickzettel, Glossar und Index.

Inhaltsverzeichnis

Vorwort 4

Für wen ist dieses Handbuch? 4

Aufbau 5

Kapitel 1: Was ist hping3? 6

1.1 Einordnung: ein Paketgenerator, kein Automatik-Scanner 6

1.2 Typische Einsatzgebiete 6

1.3 Warum „Dual-Use“? 7

Zusammenfassung 7

Kapitel 2: TCP/IP-Grundlagen für hping3 8

2.1 Der IP-Header in Kürze 8

2.2 Der TCP-Header und seine Flags 8

2.3 Portnummern, Fenstergröße und ihre Rolle 9

Zusammenfassung 9

Kapitel 3: Installation und Laborumgebung 10

3.1 Installation 10

3.2 Root-Rechte und Capabilities 10

3.3 Das isolierte Testnetz 11

Zusammenfassung 11

Kapitel 4: Grundlegende Syntax und wichtigste Optionen 12

4.1 Grundgerüst eines Aufrufs 12

4.2 Die wichtigsten TCP-Optionen 12

4.3 Zeitstempel, Wiederholungen und Datenpayload 13

Zusammenfassung 13

Kapitel 5: Portscanning mit hping3 14

5.1 Einzelne Ports gezielt prüfen 14

5.2 Mehrere Ports mit einer Schleife 14

5.3 UDP-Ports prüfen 15

Zusammenfassung 15

Kapitel 6: Firewall- und IDS-Tests 16

6.1 Die Grundidee 16

6.2 Fragmentierte Pakete testen . . . 16
6.3 Spoofed Quellports zur Regelumgehung testen . . . 16
6.4 IDS/IPS-Signaturen prüfen . . . 17
Zusammenfassung . . . 17

Kapitel 7: Idle Scan und Betriebssystem-Fingerprinting . . . 18

7.1 Das Konzept . . . 18
7.2 Betriebssystem-Fingerprinting über TTL . . . 19
7.3 Fenstergröße als zweites Indiz . . . 19
Zusammenfassung . . . 19

Kapitel 8: Denial-of-Service-Grundlagen – ausschließlich im eigenen Labor . . . 20

8.1 Das Prinzip eines SYN-Floods . . . 20
8.2 Was du dabei beobachten solltest . . . 21
8.3 Gegenmaßnahmen aus Verteidigersicht . . . 21
Zusammenfassung . . . 22

Kapitel 9: Scripting mit der eingebauten Tcl-Umgebung . . . 23

9.1 Warum eine Skriptsprache in einem Netzwerktool? . . . 23
9.2 Ein einfaches Beispiel-Skript . . . 23
9.3 Wann Shell-Schleife, wann Tcl-Skript? . . . 24
Zusammenfassung . . . 24

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen . . . 25

10.1 Laboraufbau . . . 25
10.2 Checkliste: sichere Laborumgebung . . . 25
10.3 Übungen . . . 25
Zusammenfassung . . . 28

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen . . . 29

11.1 Häufige Probleme . . . 29
11.2 Spickzettel . . . 29
11.3 Glossar . . . 30
11.4 Weiterführende Ressourcen . . . 30
Zusammenfassung . . . 30

Verzeichnis der Befehlsbeispiele . . . 32

Index . . . 33

Vorwort

`hping3` ist eines der ältesten und vielseitigsten Werkzeuge der Netzwerksicherheit: ein Kommandozeilenprogramm, mit dem du nahezu beliebige TCP-, UDP-, ICMP- und Rohpakete von Hand zusammenstellen und verschicken kannst. Während Werkzeuge wie `ping` oder `nmap` feste Abläufe automatisieren, gibt dir `hping3` die volle Kontrolle über jedes einzelne Feld eines Pakets – und damit auch das Verständnis dafür, was unter der Oberfläche von TCP/IP tatsächlich passiert.

Dieses Handbuch ergänzt das bereits vorliegende Ethical-Hacking-Handbuch um eine Tiefenbohrung zu genau diesem einen Werkzeug: Portscanning, Firewall- und IDS-Tests, Idle Scans, Betriebssystem-Fingerprinting, einfache Skripte und, mit besonderer Vorsicht behandelt, die Grundlagen von Flood-Techniken zu reinen Testzwecken im eigenen Labor.

> **ILLEGAL – niemals ohne Auftrag**
>
> Wie schon im Ethical-Hacking-Handbuch gilt hier ohne Ausnahme: Jede Technik in diesem Buch ist **ausschließlich** für den Einsatz in einer eigenen, isolierten Testumgebung oder im Rahmen eines schriftlich autorisierten Auftrags bestimmt. `hping3` kann Netzwerke spürbar belasten und Dienste stören. Entscheidend sind dabei die Berechtigung, der konkrete Zweck und der Umfang des Tests: Der Einsatz gegen fremde Systeme ohne entsprechende Berechtigung kann strafbar sein, unter anderem nach §§ 202a–202c sowie § 303b StGB (Computersabotage), und erheblichen Schaden anrichten. Teste ausschließlich Systeme, die dir gehören oder für die du eine ausdrückliche Autorisierung besitzt; bei Auftragsarbeiten müssen Umfang und Grenzen der Tests vorab eindeutig festgelegt sein (Ethical-Hacking-Handbuch, Kapitel 1 und 5).

Für wen ist dieses Handbuch?

Du bist hier richtig, wenn du

- das Ethical-Hacking-Handbuch bereits kennst und ein einzelnes Werkzeug vertiefen möchtest,
- verstehen willst, wie TCP/IP-Pakete auf Feldebene aufgebaut sind, statt es nur aus Diagrammen zu kennen,
- Firewalls, IDS/IPS oder eigene Netzwerkdienste in einem eigenen Testnetz prüfen möchtest,
- dich auf Zertifizierungen vorbereitest, in denen Paketanalyse und -erzeugung vorkommen (z. B. eJPT, OSCP, CompTIA Network+/Security+).

Aufbau

Kapitel	Thema
1	Was ist hping3? Geschichte, Einordnung, Abgrenzung zu ping und nmap
2	TCP/IP-Grundlagen, die für hping3 wichtig sind
3	Installation unter Linux, Kali und in einer eigenen Laborumgebung
4	Grundlegende Syntax und die wichtigsten Optionen
5	Portscanning mit hping3
6	Firewall- und IDS-Tests
7	Idle Scan (Zombie-Scan) und Betriebssystem-Fingerprinting
8	Denial-of-Service-Grundlagen – ausschließlich im eigenen Labor
9	Scripting mit der eingebauten Tcl-Umgebung
10	Praxisteil: Laboraufbau und geführte Übungen
11	Troubleshooting, Spickzettel, Glossar und Ressourcen

> **TIPP – Profi-Trick**
>
> Baue dir vor dem Weiterlesen ein eigenes, isoliertes Testnetz auf (Kapitel 3 und 10) – mit zwei virtuellen Maschinen im internen Netzwerk reichst du für praktisch jede Übung in diesem Buch völlig aus.

Kapitel 1: Was ist hping3?

Lernziele

- Du kannst hping3 von ping und nmap funktional abgrenzen.
- Du kennst die Geschichte und die typischen Haupteinsatzgebiete von hping3.
- Du verstehst, warum hping3 als „Dual-Use-Werkzeug“ gilt und was das für dich bedeutet.

1.1 Einordnung: ein Paketgenerator, kein Automatik-Scanner

Die meisten Netzwerkwerkzeuge entscheiden selbst, wie ein Paket aussieht. `ping` verschickt feste ICMP-Echo-Anfragen, `nmap` baut Scan-Pakete nach vordefinierten Mustern zusammen. `hping3` geht einen anderen Weg: Es ist im Kern ein **Paketgenerator**, der dir erlaubt, praktisch jedes Feld eines TCP-, UDP-, ICMP- oder Rohpakets selbst zu bestimmen – Quellport, Zielport, Flags, Sequenznummern, Fenstergröße, TTL und mehr.

Werkzeug	Grundidee	Typischer Einsatz
`ping`	feste ICMP-Echo-Anfrage	Erreichbarkeit grob prüfen
`nmap`	automatisierte Scan-Logik mit vielen Zielen	Netzwerk- und Diensterkennung in der Fläche
`hping3`	manuell konstruierte Einzelpakete	gezielte Tiefenanalyse, Firewall-Tests, Protokoll-Experimente

Gut zu wissen

Der Name ist Programm: „hping“ steht für „ping, aber mit mehr Kontrolle“. Ursprünglich 1998 von Salvatore Sanfilippo (bekannt auch als Autor von Redis) als `hping` veröffentlicht, folgte mit `hping2` und schließlich `hping3` eine Version mit eingebauter **Tcl-Skriptsprache** für komplexere Abläufe (Kapitel 9).

1.2 Typische Einsatzgebiete

- **Portscanning** jenseits von ICMP-Blockaden (Kapitel 5)
- **Firewall- und IDS/IPS-Tests**: Welche Pakettypen werden durchgelassen, welche blockiert oder protokolliert? (Kapitel 6)
- **Idle Scan**: ein Portscan, der die eigene Quelladresse verschleiert, indem ein drittes System als „Zombie“ genutzt wird (Kapitel 7)
- **Betriebssystem-Fingerprinting** über TTL-Werte, Fenstergrößen und andere Stapel-Eigenheiten (Kapitel 7)
- **Path-MTU-Discovery und Netzwerkdiagnose** jenseits der Möglichkeiten von `ping`
- **Grundlagenverständnis von TCP/IP**: Weil du jedes Feld selbst setzt, verstehst du den Drei-Wege-Handshake, Fragmentierung und Flags nach kurzer Zeit auf einer ganz anderen Ebene als aus einem Lehrbuchdiagramm

1.3 Warum „Dual-Use“?

Dieselben Fähigkeiten, die `hping3` zu einem hervorragenden Diagnosewerkzeug machen, erlauben auch das genaue Gegenteil: gezielt viele Pakete an ein System zu senden, um es zu überlasten (Kapitel 8), oder eine Firewall-Regel gezielt zu umgehen, statt sie nur zu testen.

LEGAL – nur mit Auftrag/Einwilligung

Es gibt keine Funktion in `hping3`, die „nur für legale Zwecke“ reserviert ist. Der einzige Unterschied zwischen einem Firewall-Test und einem Angriff ist – wie im gesamten Ethical-Hacking-Handbuch – die **Autorisierung**: ein schriftlicher Auftrag mit definiertem Scope oder ein System, das dir selbst gehört und vollständig vom Internet isoliert ist (Kapitel 3 und 10).

Zusammenfassung

Zusammenfassung

- hping3 erzeugt einzelne, manuell konfigurierbare Pakete, statt wie andere Werkzeuge feste Abläufe zu automatisieren.
- Haupteinsatzgebiete sind Portscanning, Firewall-Tests, Idle Scan, OS-Fingerprinting und Netzwerkdiagnose.
- Wie bei jedem Dual-Use-Werkzeug entscheidet ausschließlich die Autorisierung über Legalität, nicht die Technik.

Kapitel 2: TCP/IP-Grundlagen für hping3

Lernziele

- Du kennst die für hping3 wichtigsten Felder von IP- und TCP-Headern.
- Du verstehst TCP-Flags und den Drei-Wege-Handshake auf Feldebene.
- Du kannst erklären, wofür TTL und Fenstergröße in der Praxis genutzt werden.

2.1 Der IP-Header in Kürze

Jedes IP-Paket trägt einen Header mit festen Feldern. Für hping3 sind diese am wichtigsten:

Feld	Bedeutung	hping3-Option
Quell-IP	Absenderadresse	`-a` / `--spoof` (siehe Warnbox unten)
Ziel-IP	Empfängeradresse	letztes Argument des Aufrufs
TTL (Time to Live)	Anzahl verbleibender Hops, bevor ein Router das Paket verwirft	`-t`
Protokoll	TCP, UDP, ICMP oder roh	`-S`/`-A`/`-F`/... (TCP), `-2` (UDP), `-1` (ICMP), `-0` (roh)
Fragment-Flags	Erlaubt/verbietet Fragmentierung	`-f`

WARNUNG – Sicherheitsrisiko

`--spoof` setzt eine falsche Absenderadresse. Das ist eine der invasivsten Funktionen in hping3: Antworten gehen an die gefälschte Adresse, nicht an dich, und der Einsatz gegen fremde Infrastruktur ist unabhängig vom Zweck praktisch immer unzulässig. In diesem Buch kommt die Option nur zur reinen Begriffserklärung vor, nicht in lauffähigen Beispielen.

2.2 Der TCP-Header und seine Flags

TCP-Pakete tragen zusätzlich sechs klassische **Flags**, die den Verbindungszustand steuern:

Flag	Name	Bedeutung	hping3-Option
SYN	Synchronize	Verbindungsaufbau anfordern	`-S`
ACK	Acknowledge	Empfang bestätigen	`-A`
FIN	Finish	Verbindung ordentlich beenden	`-F`
RST	Reset	Verbindung sofort abbrechen	`-R`
PSH	Push	Daten sofort an die Anwendung weiterreichen	`-P`
URG	Urgent	Dringende Daten markieren	`-U`

Beispiel 2.1: Der TCP-Drei-Wege-Handshake in hping3-Begriffen

```
Client  --- SYN ------------------> Server      (hping3 -S)
Client  <-- SYN+ACK ----------------- Server      (Antwort, falls Port offen)
Client  --- ACK ------------------> Server      (hping3 -A, Verbindung steht)
```

Diese Flags lassen sich mit hping3 **beliebig kombinieren** oder auch **weglassen** – etwas, das ein normaler TCP-Stack so nie täte. Genau diese Freiheit macht Firewall-Tests in Kapitel 6 möglich: Eine Firewall, die nur „normale“ Handshakes erwartet, reagiert auf ein Paket mit allen Flags gesetzt oder gar keinem gesetzten Flag oft anders als erwartet.

2.3 Portnummern, Fenstergröße und ihre Rolle

- **Quell- und Zielport** (`-s`, `-p`) bestimmen, welcher Dienst angesprochen wird und wohin eine Antwort zurückgeht.
- **Fenstergröße (Window Size)** gibt an, wie viele Bytes ungestützt empfangen werden können, bevor eine Bestätigung nötig ist. Verschiedene Betriebssysteme und Netzwerkstapel nutzen charakteristische Standardwerte – eine der Grundlagen für das Fingerprinting in Kapitel 7.
- **Sequenz- und Bestätigungsnummern** (`-M`, `-L`) verfolgen, welche Daten bereits übertragen wurden.

> **Gut zu wissen**
>
> Jedes dieser Felder lässt sich mit hping3 unabhängig von den anderen setzen. Das ist der Kern dessen, was „Paketgenerator“ in Kapitel 1 bedeutet: Ein normaler TCP-Stack verknüpft diese Felder nach festen Regeln, hping3 erlaubt dir, diese Regeln bewusst zu brechen – nützlich zum Lernen und Testen, aber auch der Grund für die Vorsicht aus Kapitel 1.3.

Zusammenfassung

> **Zusammenfassung**
>
> - Die wichtigsten IP-Felder für hping3 sind Quell-/Ziel-IP, TTL, Protokoll und Fragmentierung.
> - TCP kennt sechs Flags (SYN, ACK, FIN, RST, PSH, URG), die hping3 frei kombinierbar macht.
> - Fenstergröße und TTL sind die Grundlage für spätere OS-Fingerprinting-Techniken (Kapitel 7).
> - Spoofing der Quelladresse ist technisch möglich, aber in diesem Buch bewusst nur erklärt, nie praktisch vorgeführt.

Kapitel 3: Installation und Laborumgebung

Lernziele

- Du installierst hping3 unter Kali, Debian/Ubuntu und Fedora/Arch.
- Du richtest ein isoliertes Testnetz für die Übungen in diesem Buch ein.
- Du verifizierst, dass deine Testumgebung keine Verbindung zum Internet hat.

3.1 Installation

Beispiel 3.1: hping3 installieren

```
# Kali Linux: bereits vorinstalliert
$ hping3 --version

# Debian/Ubuntu
$ sudo apt update && sudo apt install hping3

# Fedora
$ sudo dnf install hping3
# Arch Linux
$ sudo pacman -S hping
```

Gut zu wissen

Unter einigen Distributionen heißt das Paket `hping` statt `hping3`, installiert aber denselben Befehl `hping3`. Unter Windows gibt es keine offizielle, gepflegte Portierung – nutze stattdessen eine Linux-VM oder WSL mit entsprechenden Einschränkungen bei Rohsockets.

WARNUNG – Sicherheitsrisiko

hping3 wird nur noch unregelmäßig weiterentwickelt, und Distributionspakete unterscheiden sich in Build-Optionen (z. B. ob Tcl-Unterstützung aktiviert ist) und Detailverhalten einzelner Optionen. Prüfe nach der Installation mit `hping3 --version` und `hping3 --help`, welche Optionen deine konkrete Version tatsächlich anbietet, bevor du dich auf ein Beispiel aus diesem Buch verlässt.

3.2 Root-Rechte und Capabilities

Das Erzeugen roher Pakete erfordert erweiterte Rechte, weil der normale Netzwerkstapel des Betriebssystems umgangen wird.

Beispiel 3.2: hping3 mit den nötigen Rechten ausführen

```
$ sudo hping3 -S -p 80 192.168.56.102
```

TIPP – Profi-Trick

Statt dauerhaft als root zu arbeiten, kannst du dem Programm gezielt die nötige **Linux-Capability** geben (Beispiel 3.2b). Danach läuft `hping3` auch ohne `sudo` – mit genau den Rechten, die für Rohsockets nötig sind, nicht mehr.

Beispiel 3.2b: Capability statt dauerhafter root-Rechte

```
$ sudo setcap cap_net_raw+ep $(which hping3)
```

3.3 Das isolierte Testnetz

Alle Übungen in diesem Buch setzen dasselbe Labor voraus wie das Ethical-Hacking-Handbuch: zwei virtuelle Maschinen in einem **Internal Network** (nicht Host-only), ohne jede Verbindung zum Internet.

Beispiel 3.3: Minimales Testnetz für dieses Buch

```
VM 1 "Kali"           – hping3 installiert, Angreifer-Perspektive
VM 2 "Metasploitable" – Zielsystem mit mehreren offenen Diensten
Beide VMs: Netzwerkadapter auf "Internes Netzwerk" (z. B. "pentestlab"),
kein NAT, kein Bridged-Adapter, kein Host-only-Zugriff auf den Host.
```

Beispiel 3.4: Isolation vor jeder Übung verifizieren

```
$ ip route
# Keine Zeile mit "default via ..." -> kein bekannter Weg ins Internet
$ ping -c 2 192.168.56.102   # Zielsystem sollte erreichbar sein
$ ping -c 2 8.8.8.8          # sollte fehlschlagen
```

LEGAL – nur mit Auftrag/Einwilligung

Diese Checkliste gilt ausnahmslos vor jeder Übung in diesem Buch, auch wenn du „nur kurz etwas ausprobieren“ willst. Details zur Unterscheidung zwischen Host-only- und Internal-Network-Adaptern stehen im Ethical-Hacking-Handbuch, Kapitel 9.

Zusammenfassung

Zusammenfassung

- hping3 ist unter Kali vorinstalliert, auf anderen Distributionen leicht nachzuinstallieren.
- Rohsockets brauchen root-Rechte oder gezielt die Capability `cap_net_raw`.
- Jede Übung dieses Buches setzt ein isoliertes, vom Internet getrenntes Testnetz voraus – vor jeder Sitzung neu verifizieren.

Kapitel 4: Grundlegende Syntax und wichtigste Optionen

Lernziele

- Du kannst die hping3-Befehlszeile für die vier Protokollmodi aufbauen.
- Du kennst die wichtigsten Optionen für Zählung, Timing und Ausgabe.
- Du kannst einen hping3-Aufruf Baustein für Baustein lesen und selbst zusammensetzen.

4.1 Grundgerüst eines Aufrufs

Beispiel 4.1: Grundstruktur

```
$ hping3 [Protokollmodus] [Flags/Optionen] [Zieladresse]
```

Jeder Aufruf braucht genau einen **Protokollmodus** (Standard ist roh/TCP ohne Flags, wenn keiner angegeben wird):

Option	Modus
`-0` / `--rawip`	Rohes IP-Paket, Protokoll frei wählbar
`-1` / `--icmp`	ICMP-Modus (wie ein erweiterter `ping`)
`-2` / `--udp`	UDP-Modus
`-8` / `--scan`	eingebauter einfacher Portscan-Modus (Kapitel 5)
`-9` / `--listen`	Horchmodus, wartet auf bestimmte Pakete
(kein Modus)	TCP-Modus, Standard

4.2 Die wichtigsten TCP-Optionen

Option	Wirkung
`-S`	SYN-Flag setzen
`-A`	ACK-Flag setzen
`-F`	FIN-Flag setzen
`-R`	RST-Flag setzen
`-p <port>`	Zielport
`-s <port>`	Quellport (Standard: zufällig)
`-c <n>`	Anzahl der zu sendenden Pakete
`-i <intervall>` oder `--fast`/`--faster`/`--flood`	Sendegeschwindigkeit (siehe Warnbox in Kapitel 8)
`-t <ttl>`	TTL-Wert setzen
`-w <bytes>`	Fenstergröße setzen
`-V`	ausführliche Ausgabe
`-N <id>`	IP-ID-Feld setzen

Beispiel 4.2: Ein einzelnes SYN-Paket senden und die Antwort lesen

```
$ sudo hping3 -S -p 80 -c 1 192.168.56.102
HPING 192.168.56.102 (eth0 192.168.56.102): S set, 40 headers + 0 data bytes
len=46 ip=192.168.56.102 ttl=64 DF id=0 sport=80 flags=SA seq=0 win=5840 rtt=1.2 ms
--- 192.168.56.102 hping statistic ---
1 packets transmitted, 1 packets received, 0% packet loss
```

`flags=SA` in der Antwort bedeutet SYN+ACK gesetzt – das ist ein starkes Indiz für einen offenen Port. Ein `flags=RA` (RST+ACK) deutet auf einen erreichbaren, aber geschlossenen Port hin. Bleibt die Antwort ganz aus, ist das **mehrdeutig**: Eine Firewall kann das Paket stillschweigend verwerfen, aber auch gewöhnlicher Paketverlust, ein falsches Routing oder ein offline befindlicher Host erklären eine ausbleibende Antwort – keine dieser drei Reaktionen liefert einen absolut sicheren Beweis, sondern jeweils nur einen wahrscheinlichen Hinweis.

Antwortmuster	Wahrscheinliche Bedeutung
`flags=SA`	Port wahrscheinlich offen
`flags=RA`	Port wahrscheinlich geschlossen, aber erreichbar
keine Antwort (Timeout)	keine eindeutige Aussage möglich: Firewall-Filterung, Paketverlust, falsches Routing oder nicht erreichbarer Host kommen alle infrage
ICMP „Port unreachable“ (bei UDP)	Port wahrscheinlich geschlossen

TIPP – Profi-Trick

`-c 1` begrenzt jeden Testaufruf in diesem Kapitel bewusst auf ein einziges Paket. Das reicht für die Analyse völlig aus und vermeidet unnötigen Datenverkehr – eine Gewohnheit, die sich auch in Kapitel 5 und 6 auszahlt.

4.3 Zeitstempel, Wiederholungen und Datenpayload

Beispiel 4.3: Nützliche Zusatzoptionen

```
$ sudo hping3 -S -p 443 -c 3 -i u200000 192.168.56.102   # 3 Pakete, 200 ms Abstand
$ sudo hping3 -S -p 80 -c 1 -d 50 192.168.56.102          # 50 Byte Nutzlast anhängen
$ sudo hping3 -S -p 80 -c 1 --tcp-timestamp 192.168.56.102  # TCP-Zeitstempel mitsenden
```

`-i u200000` gibt das Intervall in Mikrosekunden an (hier 200 Millisekunden). Ohne `-i` sendet hping3 standardmäßig im Sekundentakt – ausreichend langsam für die meisten Lernübungen in diesem Buch.

Zusammenfassung

Zusammenfassung

- Ein hping3-Aufruf besteht aus Protokollmodus, Flags/Optionen und Zieladresse.
- `-S`, `-A`, `-F`, `-R` setzen TCP-Flags; `-p`/`-s` legen Ziel- und Quellport fest.
- Die Antwortflags (`SA`, `RA`, Timeout) geben einen wahrscheinlichen, aber keinen absolut sicheren Hinweis darauf, ob ein Port offen, geschlossen oder gefiltert ist – besonders ein Ausbleiben der Antwort lässt mehrere Erklärungen zu.
- `-c 1` und moderate Intervalle sind für Lernzwecke der sinnvolle Standard.

Kapitel 5: Portscanning mit hping3

Lernziele

- Du führst einzelne und mehrfache Portprüfungen mit hping3 durch.
- Du nutzt den eingebauten Scan-Modus für mehrere Ports gleichzeitig.
- Du verstehst, wann hping3 eine sinnvolle Ergänzung zu Nmap ist – und wann nicht.

ILLEGAL – niemals ohne Auftrag

Ausschließlich gegen das eigene, isolierte Testnetz aus Kapitel 3 oder im Rahmen eines autorisierten Auftrags (Ethical-Hacking-Handbuch, Kapitel 1).

5.1 Einzelne Ports gezielt prüfen

Beispiel 5.1: Einen einzelnen Port testen

```
$ sudo hping3 -S -p 22 -c 1 192.168.56.102
len=46 ... flags=SA ... -> Port 22 offen

$ sudo hping3 -S -p 12345 -c 1 192.168.56.102
len=46 ... flags=RA ... -> Port 12345 geschlossen
```

5.2 Mehrere Ports mit einer Schleife

hping3 scannt von sich aus nur einen Port pro Aufruf (abgesehen vom eingebauten Scan-Modus, siehe 5.3). Für mehrere Ports kombinierst du es mit einer einfachen Shell-Schleife:

Beispiel 5.2: Mehrere Ports nacheinander prüfen

```
$ for port in 21 22 23 80 443 3306; do
    echo "Port $port:"
    sudo hping3 -S -p $port -c 1 192.168.56.102 2>&1 | grep flags
  done
```

Beispiel 5.3: Portbereich mit --scan

```
$ sudo hping3 --scan 1-1000 -S 192.168.56.102
Scanning 192.168.56.102 (192.168.56.102), port 1-1000
1000 ports to scan, use -V to see all the replies
+----+-----------+---------+---+-----+-----+-----+
|port| serv name |  flags  |ttl| id  | win | len |
+----+-----------+---------+---+-----+-----+-----+
  21   ftp         : .S..A..        64   0    5840   46
  22   ssh         : .S..A..        64   0    5840   46
  80   http        : .S..A..        64   0    5840   46
All replies received. Done.
Not responding ports: ...
```

Gut zu wissen

Der eingebaute `--scan`-Modus ist bequem, aber bewusst einfach gehalten: Er kennt keine Service-Versionserkennung und keine Skript-Engine wie Nmap. Für eine erste, flexible Analyse einzelner, interessanter Ports ist hping3 ideal; für eine vollständige Netzwerk- und Diensterkennung in der Fläche bleibt Nmap (Ethical-Hacking-Handbuch, Kapitel 4) das richtige Werkzeug. Viele Pentester:innen nutzen beide: Nmap für die Übersicht, hping3 für die gezielte Tiefenanalyse einzelner auffälliger Ports.

5.3 UDP-Ports prüfen

Beispiel 5.4: Einen UDP-Port testen

```
$ sudo hping3 -2 -p 53 -c 1 192.168.56.102
# Keine Antwort: Port offen, durch Firewall gefiltert, ODER Paket unterwegs verloren
# (UDP ist "stumm" und liefert daher die am wenigsten eindeutigen Ergebnisse)
# ICMP "Port Unreachable": Port mit hoher Wahrscheinlichkeit geschlossen
```

UDP liefert grundsätzlich weniger eindeutige Ergebnisse als TCP, weil ein offener UDP-Port oft überhaupt nicht antwortet – das Fehlen einer Antwort ist mehrdeutig (siehe Tabelle in Kapitel 4.2).

Zusammenfassung

Zusammenfassung

- Einzelne Ports lassen sich mit `-S -p <port> -c 1` gezielt prüfen; die Antwortflags liefern einen wahrscheinlichen Hinweis, keine absolute Gewissheit.
- Für mehrere Ports kombiniert man hping3 entweder mit einer Shell-Schleife oder nutzt `--scan`.
- hping3 ergänzt Nmap für die gezielte Tiefenanalyse, ersetzt es aber nicht für die flächendeckende Erkennung.
- UDP-Scans sind grundsätzlich mehrdeutiger als TCP-Scans.

Kapitel 6: Firewall- und IDS-Tests

Lernziele

- Du testest mit hping3 gezielt, welche Pakettypen eine Firewall durchlässt.
- Du nutzt ungewöhnliche Flag-Kombinationen, um Filterregeln sichtbar zu machen.
- Du verstehst, wie sich dieselben Techniken auch zur IDS/IPS-Signaturprüfung einsetzen lassen.

ILLEGAL – niemals ohne Auftrag

Ausschließlich gegen das eigene, isolierte Testnetz oder im Rahmen eines autorisierten Auftrags.

6.1 Die Grundidee

Eine Firewall-Regel beschreibt, welche Pakete erlaubt sind – nach Quelle, Ziel, Port und oft auch Pakettyp. hping3 lässt dich genau diese Variablen systematisch durchspielen und beobachten, wo die Regel greift.

Beispiel 6.1: Denselben Port mit unterschiedlichen Flags testen

```
$ sudo hping3 -S -p 80 -c 1 192.168.56.102    # normales SYN
$ sudo hping3 -F -p 80 -c 1 192.168.56.102    # nur FIN gesetzt
$ sudo hping3 -A -p 80 -c 1 192.168.56.102    # nur ACK gesetzt
$ sudo hping3 -SF -p 80 -c 1 192.168.56.102   # SYN und FIN gleichzeitig (unüblich)
```

Eine Firewall, die nur den Beginn eines normalen Handshakes (SYN) erwartet, reagiert auf diese vier Varianten oft unterschiedlich: Manche lassen ein einzelnes FIN-Paket unbeachtet durch, weil sie nur SYN-Pakete als „neue Verbindung“ protokollieren – ein klassischer, gut dokumentierter Testfall für zustandsbehaftete (Stateful) versus zustandslose (Stateless) Filterregeln.

6.2 Fragmentierte Pakete testen

Beispiel 6.2: Fragmentierung erzwingen

```
$ sudo hping3 -S -p 80 -c 1 -f 192.168.56.102
```

`-f` erzwingt Fragmentierung, selbst wenn das Paket eigentlich klein genug für ein einzelnes Stück wäre. Manche ältere oder schlecht konfigurierte Firewalls prüfen nur das erste Fragment auf verbotene Inhalte und lassen nachfolgende Fragmente unkontrolliert durch – ein historisch bekanntes, heute meist behobenes Einfallstor, das sich im Labor gut nachvollziehen lässt.

6.3 Spoofed Quellports zur Regelumgehung testen

Firewalls vertrauen mitunter bestimmten Quellports (z. B. Port 53 für DNS-Antworten) besonders stark.

Beispiel 6.3: Mit einem vertrauenswürdig wirkenden Quellport testen

```
$ sudo hping3 -S -p 22 -s 53 -c 1 192.168.56.102
```

Antwortet das Zielsystem hier anders als bei einem zufälligen Quellport, deutet das auf eine Regel hin, die Quellport 53 pauschal bevorzugt – ein in der Praxis dokumentierter Fehlkonfigurationstyp.

6.4 IDS/IPS-Signaturen prüfen

Ein IDS/IPS erkennt Angriffe oft an charakteristischen Mustern (z. B. einer bestimmten Flag-Kombination oder Paketgröße). Mit hping3 lässt sich im eigenen Labor nachvollziehen, ob eine Signatur tatsächlich wie erwartet auslöst:

Beispiel 6.4: Ein bekanntes Testmuster senden und im IDS-Log prüfen

```
$ sudo hping3 -S -p 80 -c 1 --tcp-timestamp -w 0 192.168.56.102
# Anschließend im Log des IDS/IPS (z. B. Suricata, Snort) prüfen,
# ob und wie das Paket erfasst wurde.
```

TIPP – Profi-Trick

Ein guter Firewall-/IDS-Test verändert immer **nur eine Variable auf einmal** (nur das Flag, nur den Quellport, nur die Fragmentierung) – genau wie beim systematischen Debuggen von Prompts oder Code. So lässt sich die Ursache einer beobachteten Reaktion eindeutig zuordnen.

Zusammenfassung

Zusammenfassung

- hping3 macht Firewall-Regeln sichtbar, indem es systematisch einzelne Pakettyp-Variablen verändert.
- Ungewöhnliche Flag-Kombinationen und erzwungene Fragmentierung decken typische Fehlkonfigurationen auf.
- Spoofed, vertrauenswürdig wirkende Quellports testen, ob Firewalls bestimmten Ports blind vertrauen.
- Dieselben Techniken eignen sich, um IDS/IPS-Signaturen im eigenen Labor zu verifizieren.

Kapitel 7: Idle Scan und Betriebssystem-Fingerprinting

Lernziele

- Du verstehst das Konzept des Idle Scans und warum er die eigene Quelladresse verschleiert.
- Du erkennst die Voraussetzungen, die ein „Zombie“-Host dafür erfüllen muss.
- Du nutzt TTL und Fenstergröße zur groben Betriebssystem-Einschätzung.

ILLEGAL – niemals ohne Auftrag

Ausschließlich im eigenen, vollständig kontrollierten Testnetz – ein Idle Scan betrifft immer **drei** Systeme (Angreifer, Zombie, Ziel), weshalb hier besondere Sorgfalt gilt, dass alle drei unter eigener Kontrolle stehen.

7.1 Das Konzept

Ein **Idle Scan** nutzt einen dritten, möglichst „ruhigen“ Host (den sogenannten **Zombie**) als Werkzeug, um einen Portscan durchzuführen, ohne dass das Zielsystem die tatsächliche Quelladresse des Scans sieht.

Beispiel 7.1: Idle Scan in drei Schritten (Konzept)

```
1. Die IP-ID-Sequenz des Zombies wird beobachtet (steigt bei jedem
   gesendeten Paket meist vorhersagbar um 1).
2. Ein Paket wird an das Ziel gesendet, dessen Absenderadresse auf den
   Zombie gefälscht ist.
3. Reagiert das Ziel (z. B. weil der Port offen ist), sendet es eine
   Antwort an den Zombie, der daraufhin selbst ein weiteres Paket
   verschickt und so seine IP-ID erneut erhöht.
4. Ein erneuter Blick auf die IP-ID-Sequenz des Zombies verrät, ob
   Schritt 3 stattgefunden hat – und damit, ob der Port offen war.
```

Beispiel 7.2: Idle Scan mit hping3 vorbereiten (IP-ID eines Zombies beobachten)

```
$ sudo hping3 -S -p 80 -c 3 192.168.56.50
len=46 ... id=15234 ...
len=46 ... id=15235 ...
len=46 ... id=15236 ...
# Eine saubere, um 1 steigende Folge zeigt: dieser Host eignet sich als Zombie.
```

Gut zu wissen

Nmap bietet mit `-sI` eine vollständig automatisierte Idle-Scan-Funktion, die genau dieses Prinzip umsetzt. hping3 eignet sich hervorragend, um das zugrunde liegende Prinzip Schritt für Schritt nachzuvollziehen, bevor man die automatisierte Variante einsetzt – das Verständnis, warum ein Zombie eine vorhersagbare IP-ID-Sequenz und wenig eigenen Netzwerkverkehr haben muss, bleibt in beiden Fällen gleich.

7.2 Betriebssystem-Fingerprinting über TTL

Jedes Betriebssystem startet mit einem charakteristischen Standard-TTL-Wert, der auf dem Weg durch Router um 1 pro Hop sinkt.

Beobachteter TTL (ungefähr)	Wahrscheinliches Ausgangssystem
nahe 64	Linux, viele BSD-Systeme
nahe 128	Windows
nahe 255	manche Netzwerkgeräte, ältere Unix-Systeme

Beispiel 7.3: TTL einer Antwort auswerten

```
$ sudo hping3 -S -p 80 -c 1 192.168.56.102
len=46 ... ttl=64 ...   -> Ausgangswert vermutlich 64, Linux-artiges System
```

WARNUNG – Sicherheitsrisiko

TTL-Fingerprinting ist eine **grobe Schätzung**, keine verlässliche Identifikation. Firewalls und Lastverteiler verändern TTL-Werte mitunter, und der beobachtete Wert verrät nur den TTL **nach** allen durchlaufenen Hops, nicht den tatsächlichen Startwert – du musst die Hop-Anzahl kennen oder schätzen, um zurückzurechnen.

7.3 Fenstergröße als zweites Indiz

Zusätzlich zur TTL unterscheiden sich Betriebssysteme in ihrer Standard-Fenstergröße. Die Kombination aus TTL und Fenstergröße liefert einen deutlich zuverlässigeren Hinweis als TTL allein – das Grundprinzip, auf dem auch automatisierte Fingerprinting-Werkzeuge wie `nmap -O` aufbauen, nur mit einer größeren, gepflegten Signaturdatenbank im Hintergrund.

Zusammenfassung

Zusammenfassung

- Ein Idle Scan verschleiert die eigene Quelladresse, indem er die IP-ID-Sequenz eines dritten „Zombie“-Hosts auswertet.
- Ein geeigneter Zombie hat wenig eigenen Verkehr und eine vorhersagbare, meist um 1 steigende IP-ID-Folge.
- TTL- und Fenstergrößen-Werte liefern grobe, aber nützliche Hinweise auf das Zielbetriebssystem.
- Automatisierte Werkzeuge wie Nmap nutzen dieselben Prinzipien mit größerer Signaturdatenbank.

Kapitel 8: Denial-of-Service-Grundlagen – ausschließlich im eigenen Labor

Lernziele

- Du verstehst das Prinzip eines SYN-Floods auf konzeptioneller Ebene.
- Du kannst die Resilienz eines eigenen Testsystems gegen hohe Paketraten einschätzen.
- Du kennst die gängigen Gegenmaßnahmen (SYN Cookies, Rate Limiting) aus Verteidigersicht.

ILLEGAL – niemals ohne Auftrag

Dieses Kapitel ist das rechtlich und ethisch heikelste des gesamten Buches. Es behandelt Techniken, die reale Dienste lahmlegen können. Jede Anwendung außerhalb eines vollständig isolierten, eigenen Testnetzes oder eines ausdrücklich für Lasttests autorisierten Auftrags ist nach § 303b StGB (Computersabotage) strafbar – unabhängig von Absicht oder angeblich „geringer" Intensität. Dieses Buch zeigt bewusst keine Optimierungen für Reichweite, Verschleierung oder Verstärkung einer solchen Last; es erklärt das Prinzip, damit du es als Verteidiger:in erkennst und testest.

8.1 Das Prinzip eines SYN-Floods

Ein **SYN-Flood** nutzt aus, dass ein Server nach Erhalt eines SYN-Pakets Ressourcen reserviert (einen Eintrag in seiner „Backlog"-Warteschlange), um auf die restlichen Schritte des Drei-Wege-Handshakes zu warten. Werden sehr viele SYN-Pakete gesendet, ohne den Handshake jemals abzuschließen, kann diese Warteschlange volllaufen, sodass legitime Verbindungsversuche abgewiesen werden.

Beispiel 8.1: Das Grundmuster eines SYN-Floods (Konzept)

```
Angreifer  --- SYN --------------> Server   (reserviert Ressource 1)
Angreifer  --- SYN --------------> Server   (reserviert Ressource 2)
Angreifer  --- SYN --------------> Server   (reserviert Ressource 3)
...        (kein ACK folgt jemals, Warteschlange läuft voll)
```

hping3 bietet dafür zwei grundsätzlich unterschiedliche Geschwindigkeitsstufen – es ist wichtig, sie nicht zu verwechseln.

8.1.1 Kontrollierter, ratenbegrenzter Test (empfohlener Einstieg)

Beispiel 8.1b: Ratenbegrenzter SYN-Test – ausschließlich gegen die eigene Testumgebung

```
$ sudo hping3 -S -p 80 -i u1000 -c 2000 192.168.56.102
```

`-i u1000` gibt ein festes Intervall von 1000 Mikrosekunden (1 Millisekunde) zwischen den Paketen vor, `-c 2000` begrenzt den gesamten Lauf auf 2000 Pakete – bei diesem Intervall also auf rund zwei Sekunden. Das ist die Variante, mit der du in Übung 7 (Kapitel 10) beginnen solltest: Rate und Dauer sind von vornherein bekannt und einstellbar, nicht nur durch beherztes `Strg+C` begrenzbar.

8.1.2 Die ungebremste Variante: --flood

Beispiel 8.1c: Ungebremster SYN-Flood – ausschließlich gegen die eigene Testumgebung, mit besonderer Vorsicht

```
$ sudo hping3 -S -p 80 --flood 192.168.56.102
```

> **WARNUNG – Sicherheitsrisiko**
>
> `--flood` ist **kein moderater Test**, sondern sendet so schnell, wie das System es zulässt, und **ignoriert dabei eingehende Antworten vollständig** – das unterscheidet es grundlegend von `-i`. Es gibt hier keine eingebaute Bremse und ohne `-c` auch keine automatische Begrenzung der Paketanzahl; nur `Strg+C` beendet den Lauf. Setze `--flood` deshalb nur ein, wenn du den ratenbegrenzten Test aus 8.1.1 bereits erfolgreich und kontrolliert durchgeführt hast, und auch dann nur für wenige Sekunden am Stück.

> **ILLEGAL – niemals ohne Auftrag**
>
> Führe beide Varianten ausschließlich gegen eine IP-Adresse aus, die **dir selbst gehört und vollständig isoliert ist** (Kapitel 3.3) – niemals gegen eine Adresse aus deinem Heimnetz, einem Arbeitsnetz oder dem Internet, auch nicht testweise für wenige Sekunden. Welche konkrete Kombination aus Zieladresse, Rate und Dauer zu einer spürbaren Belastung führt, hängt vollständig von der Kapazität des Testsystems ab; steigere deshalb nur schrittweise und beobachte dabei durchgehend das Zielsystem.

> **WARNUNG – Sicherheitsrisiko**
>
> Selbst der ratenbegrenzte Test kann im eigenen Labor die Test-VM, den virtuellen Switch oder in ungünstigen Fällen den Host-Rechner spürbar belasten – bei `--flood` gilt das umso mehr. Beobachte die Systemauslastung auf dem Zielsystem durchgehend (z. B. mit `top` oder `htop`) und beginne in jedem Fall mit der kleinsten, kürzesten Variante.

8.2 Was du dabei beobachten solltest

Für die erste Zeile der folgenden Tabelle zählst du die Anzahl der Verbindungen im Zustand SYN_RECV, zum Beispiel mit `ss -tan state syn-recv`, gefolgt von einer Zeilenzählung.

Messgröße	Werkzeug auf dem Zielsystem	Was ein Problem anzeigt
Offene Verbindungen im SYN_RECV-Zustand	`ss` mit Filter auf SYN_RECV, Zeilen gezählt	ein stark ansteigender Wert
CPU- und Netzwerklast	`top`, `htop`, `iftop`	ungewöhnlich hohe Auslastung bei geringer Nutzlast
Erreichbarkeit für einen regulären Client	eine dritte VM, die normal verbindet	spürbare Verzögerung oder Abbrüche

8.3 Gegenmaßnahmen aus Verteidigersicht

- **SYN Cookies**: Der Server reserviert keine Ressource für ein SYN-Paket, sondern kodiert den nötigen Zustand direkt in die Sequenznummer der Antwort – unter Linux mit `net.ipv4.tcp_syncookies` steuerbar.
- **Rate Limiting**: Firewalls und Lastverteiler begrenzen die Anzahl neuer Verbindungen pro Quelladresse und Zeiteinheit.

- **Backlog-Größe erhöhen**: Ein größerer Warteschlangenpuffer verschiebt die Schwelle, ab der ein Flood wirkt, verhindert sie aber nicht grundsätzlich.
- **Vorgelagerte Schutzdienste**: In der Praxis übernehmen spezialisierte Dienste (z. B. bei Hosting-Anbietern) die Filterung großer Angriffsvolumen, bevor sie das eigentliche System erreichen.

TIPP – Profi-Trick

Der lehrreichste Versuch in diesem Kapitel ist nicht der Flood selbst, sondern der **Vorher-Nachher-Vergleich**: Aktiviere SYN Cookies auf deinem Testsystem, wiederhole denselben kurzen, moderaten Test und vergleiche die Beobachtungen aus Tabelle 8.2.

Zusammenfassung

Zusammenfassung

- Ein SYN-Flood nutzt aus, dass Server für jedes SYN-Paket Ressourcen reservieren, bevor der Handshake abgeschlossen ist.
- Jeder Test gehört ausschließlich in ein eigenes, isoliertes Labor, beginnend mit sehr kurzer Dauer und schrittweiser Steigerung.
- SYN-Zustände, CPU-Last und Erreichbarkeit von einer dritten VM aus sind die wichtigsten Messgrößen.
- SYN Cookies und Rate Limiting sind die zentralen Gegenmaßnahmen und lassen sich im selben Labor direkt vergleichen.

Kapitel 9: Scripting mit der eingebauten Tcl-Umgebung

Lernziele

- Du kennst den Zweck der in hping3 eingebauten Tcl-Skriptumgebung.
- Du schreibst ein einfaches Skript, das mehrere Pakete automatisiert auswertet.
- Du weißt, wann sich ein Skript lohnt und wann eine einfache Shell-Schleife genügt.

9.1 Warum eine Skriptsprache in einem Netzwerktool?

Für einfache, wiederholte Aufrufe reichen Shell-Schleifen (Kapitel 5.2) völlig aus. Sobald du aber auf einzelne Antwortfelder reagieren möchtest – etwa „sende das nächste Paket nur, wenn das vorherige eine bestimmte Flag-Kombination zurückgab“ –, stößt eine reine Shell-Schleife an ihre Grenzen. Genau dafür bringt hping3 eine eingebaute **Tcl**-Umgebung mit.

Beispiel 9.1: Interaktiven Tcl-Modus starten

```
$ hping3
hping3>
```

Es genügt, `hping3` **ohne jede Option** aufzurufen: Wurde das Programm mit Tcl-Unterstützung übersetzt (bei den meisten Distributionspaketen der Fall), öffnet sich statt eines einzelnen Sende-Vorgangs eine interaktive `hping3>`-Eingabeaufforderung. Mit `exit` verlässt du sie wieder.

WARNUNG – Sicherheitsrisiko

Es gibt **keine** Option `--tcl`. Prüfe bei Unsicherheit mit `hping3 --help` oder `man hping3`, ob deine installierte Version wie beschrieben reagiert – das genaue Verhalten kann je nach Distribution und Build-Konfiguration leicht abweichen.

Ein fertiges Skript führst du dagegen **nicht** interaktiv aus, sondern direkt von der Shell aus mit dem Unterbefehl `exec`:

Beispiel 9.1b: Ein Tcl-Skript ausführen

```
$ hping3 exec mein_skript.htcl
```

9.2 Ein einfaches Beispiel-Skript

Beispiel 9.2: hping-Send/Receive-Zyklus in Tcl (vereinfachtes Demonstrationsbeispiel)

```
hping3> hping send {ip(daddr=192.168.56.102)+tcp(flags=s,dport=80)}
hping3> set antwort [hping recv eth0]
hping3> puts "Erhalten: $antwort"
```

Gut zu wissen

Dies ist ein **stark vereinfachtes Demonstrationsbeispiel**, kein einsatzfertiges Muster. `hping recv` liefert das nächste auf dem angegebenen Interface eintreffende Paket zurück – ohne weitere Filterung ist das nicht zwangsläufig die Antwort auf das zuvor gesendete SYN, sondern könnte jedes andere Paket auf diesem Interface sein. In echten Skripten filterst du die Antwort gezielt (z. B. nach Quelladresse und Ports) und begrenzt die Wartezeit, sofern deine hping3-Version das unterstützt; andernfalls kann der Aufruf unbegrenzt auf ein passendes Paket warten. Details dazu stehen in der Tcl-API-Dokumentation deiner hping3-Installation (`docs/API.txt` im Quellcode).

Das Grundprinzip bleibt dennoch gültig: `hping send` baut ein Paket aus benannten Parametern zusammen und verschickt es, `hping recv` liest ein Paket auf dem angegebenen Interface. Darauf lassen sich beliebige Tcl-Kontrollstrukturen (Schleifen, Bedingungen, Dateiausgabe) aufsetzen.

Gut zu wissen

Die genaue Tcl-Syntax von hping3 ist in der Praxis selten nötig: Für die meisten Aufgaben in diesem Buch reichen einzelne Aufrufe oder einfache Shell-Schleifen. Die Skriptumgebung lohnt sich vor allem für wiederkehrende, komplexere Testabläufe, etwa automatisierte Firewall-Regressionstests nach jeder Konfigurationsänderung. Teste jedes Skript zunächst in deiner eigenen Laborumgebung, bevor du dich auf sein Verhalten verlässt.

9.3 Wann Shell-Schleife, wann Tcl-Skript?

Situation	Empfehlung
Ein paar Ports nacheinander prüfen	Shell-Schleife (Beispiel 5.2)
Fester Testablauf, der oft wiederholt wird	Tcl-Skript, einmal geschrieben, immer wieder ausführbar
Reaktion auf konkrete Antwortinhalte nötig	Tcl-Skript (Shell-Schleifen können das nur umständlich)
Ergebnisse sollen strukturiert geloggt werden	Tcl-Skript mit Dateiausgabe, oder hping3-Ausgabe in eine Datei umleiten und separat auswerten

Zusammenfassung

Zusammenfassung

- hping3 bringt eine eingebaute Tcl-Umgebung für Abläufe mit, die auf einzelne Antwortinhalte reagieren müssen.
- `hping send` und `hping recv` sind die zentralen Bausteine eines Skripts.
- Für die meisten Aufgaben in diesem Buch reichen einzelne Aufrufe oder Shell-Schleifen; Tcl lohnt sich bei wiederkehrenden, komplexeren Testabläufen.

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen

> **Lernziele**
>
> - Du baust das für dieses Buch nötige Labor auf.
> - Du arbeitest acht Übungen durch, die alle vorherigen Kapitel praktisch verbinden.
> - Du weißt, dass abweichende Ergebnisse nicht automatisch ein Fehler sind, sondern oft an Betriebssystem, Kernel- oder Firewall-Konfiguration liegen.

10.1 Laboraufbau

Dieses Buch nutzt dasselbe Labor-Grundgerüst wie das Ethical-Hacking-Handbuch (dort Kapitel 9, mit ausführlichen Installationsschritten für VirtualBox, Kali und Metasploitable). Für die Übungen unten reichen:

- **Kali-VM** mit installiertem `hping3` (Kapitel 3.1)
- **Metasploitable-2-VM** oder eine andere, bewusst einfach konfigurierte Linux-Test-VM als Ziel
- Beide im selben **Internal Network**, ohne Internetzugang (Kapitel 3.3)
- Optional: eine dritte, einfache VM mit aktivierbarer Firewall (z. B. `iptables`/`nftables`) für Kapitel 6

10.2 Checkliste: sichere Laborumgebung

Arbeite diese Punkte vor **jeder** Übung durch, auch wenn du „nur kurz“ etwas ausprobieren willst – besonders vor den Übungen 7 und 8.

Checkliste 10.1: Vor jeder Übung

```
[ ] Netzwerkadapter aller beteiligten VMs steht auf "Internes Netzwerk"
[ ] `ip route` zeigt keine Standardroute ins Internet (Beispiel 3.4)
[ ] Zielsystem ist eine VM, die dir selbst gehört, kein reales/fremdes System
[ ] Für Übung 7/8: Snapshot der Ziel-VM vorhanden, Systemmonitor (top/htop)
    auf der Ziel-VM bereits geöffnet, bevor der Test beginnt
```

10.3 Übungen

Jede Übung folgt demselben Schema: Ziel, Voraussetzungen, Durchführung, erwartetes Ergebnis, Auswertung und – falls dein Ergebnis abweicht – Fehlerbehebung. Eine Abweichung vom „erwarteten Ergebnis“ ist dabei kein Beinbruch: Betriebssystemversion, Kernel-Parameter und Firewall-Regeln beeinflussen das tatsächliche Verhalten spürbar.

Übung 1: Erste Pakete senden und Antworten lesen

Ziel: Die Grundbedeutung der Antwortflags `SA` und `RA` in der Praxis nachvollziehen. **Voraussetzungen:** Kali-VM mit hping3, erreichbare Metasploitable-VM (Kapitel 3). **Durchführung:** Sende je ein SYN-Paket an einen bekannt offenen Port (z. B. 22) und einen beliebigen, voraussichtlich geschlossenen Port (z. B. 12345) deiner Metasploitable-VM (Beispiel 5.1). **Erwartetes Ergebnis:** Der offene Port antwortet mit `flags=SA`, der geschlossene mit `flags=RA`. **Auswertung:** `SA` deutet auf einen offenen, `RA` auf einen erreichbaren, aber geschlossenen Port hin (Kapitel 4.2) – beides sind wahrscheinliche, keine absolut sicheren Aussagen. **Fehlerbehebung:** Erhältst du bei beiden Ports keine Antwort, prüfe zuerst die Checkliste 10.1 (Erreichbarkeit, Netzwerkadapter), bevor du von einer Firewall ausgehst.

Übung 2: Mehrere Ports per Schleife prüfen

Ziel: Mehrere Ports systematisch erfassen und mit Nmap gegenprüfen. **Voraussetzungen:** wie Übung 1, zusätzlich installiertes Nmap. **Durchführung:** Erweitere Beispiel 5.2 um mindestens zehn Ports deiner Wahl und erzeuge eine kurze Tabelle „Port – Status“. Wiederhole dieselben Ports danach mit `nmap -p <liste> <ziel>`. **Erwartetes Ergebnis:** hping3 und Nmap kommen für dieselben Ports zum selben Befund (offen/geschlossen). **Auswertung:** Stimmen beide überein, bestätigt das deine Interpretation der hping3-Antworten. **Fehlerbehebung:** Bei Abweichungen einzelner Ports: Wiederhole den hping3-Test mit `-c 3` statt `-c 1` – einzelne Pakete können unterwegs verloren gehen, ohne dass das etwas mit dem Portstatus zu tun hat.

Übung 3: TTL-Schätzung

Ziel: Die Grenzen von TTL-basiertem Fingerprinting selbst erleben. **Voraussetzungen:** wie Übung 1. **Durchführung:** Sende ein einzelnes SYN-Paket an deine Ziel-VM, lies den TTL-Wert der Antwort und vergleiche ihn mit der Tabelle in Kapitel 7.2. Prüfe danach auf der Ziel-VM selbst mit `cat /proc/sys/net/ipv4/ip_default_ttl` den tatsächlichen Ausgangswert. **Erwartetes Ergebnis:** Bei einer Linux-Ziel-VM im selben Subnetz (ein Hop) liegt der beobachtete Wert typischerweise nahe am tatsächlichen Ausgangswert, meist 64. **Auswertung:** Die Differenz zwischen beobachtetem und tatsächlichem Wert entspricht der Anzahl der Hops dazwischen – im selben Subnetz idealerweise 0 oder 1. **Fehlerbehebung:** Größere Abweichungen deuten auf zusätzliche Hops (z. B. einen virtuellen Router) zwischen den VMs hin; das ist kein Fehler, sondern eine reale Eigenschaft deiner Netzwerktopologie.

Übung 4: Firewall-Regel sichtbar machen

Ziel: Beobachten, wie unterschiedliche TCP-Flags auf eine einfache Firewall-Regel treffen. **Voraussetzungen:** eine dritte Test-VM mit `iptables`/`nftables`. **Durchführung:** Aktiviere eine Regel, die nur SYN-Pakete auf Port 80 erlaubt und alles andere verwirft. Teste danach SYN, FIN und ACK einzeln (Beispiel 6.1). **Erwartetes Ergebnis:** Das SYN-Paket erhält eine Antwort; FIN und ACK bleiben wahrscheinlich unbeantwortet oder werden je nach Regel sogar verworfen. **Auswertung:** Notiere, welche Flag-Variante welches Verhalten auslöst – das ist die Grundlage für Übung 5. **Fehlerbehebung:** Reagiert die Firewall auf alle Varianten gleich, prüfe mit `iptables -L -v` (bzw. dem nftables-Äquivalent), ob die Regel tatsächlich aktiv und an der richtigen Kette hinterlegt ist.

Übung 5: Fragmentierung testen

Ziel: Prüfen, ob erzwungene Fragmentierung eine bestehende Firewall-Regel umgeht. **Voraussetzungen:** dieselbe Firewall-Regel wie in Übung 4. **Durchführung:** Wiederhole Übung 4 mit erzwungener Fragmentierung (`-f`, Beispiel 6.2). **Erwartetes Ergebnis:** Mit einer modernen, korrekt konfigurierten Regel ändert sich das Verhalten gegenüber Übung 4 nicht. **Auswertung:** Kein Unterschied ist hier das „gute“ Ergebnis; ein Unterschied wäre ein Hinweis auf eine Lücke in der Regel. **Fehlerbehebung:** Beobachtest du einen Unterschied, prüfe zuerst, ob deine Firewall-Regel überhaupt Fragment-Behandlung konfiguriert hat (z. B. `--fragment` bei älteren `iptables`-Syntaxen), bevor du eine echte Lücke vermutest.

Übung 6: Zombie-Eignung prüfen

Ziel: Die Voraussetzungen für einen geeigneten Idle-Scan-Zombie praktisch einschätzen. **Voraussetzungen:** eine dritte, weitgehend untätige Test-VM. **Durchführung:** Sende drei SYN-Pakete an diese VM (Beispiel 7.2) und notiere die IP-ID-Werte der Antworten. **Erwartetes Ergebnis:** Eine Folge wie 15234, 15235, 15236 (Schritt von 1). **Auswertung:** Eine solche, gleichmäßig um 1 steigende Folge zeigt eine gute Zombie-Eignung; unregelmäßige Sprünge deuten auf einen Host mit viel eigenem Netzwerkverkehr hin. **Fehlerbehebung:** Beobachtest du unregelmäßige Werte, prüfe, ob auf der VM im Hintergrund andere Netzwerkdienste oder automatische Updates laufen.

Übung 7: SYN-Test-Resilienz beobachten (kontrollierte Variante)

Ziel: Die Wirkung einer definierten, ratenbegrenzten Paketlast auf die SYN-Warteschlange messen. **Voraussetzungen:** Checkliste 10.1 vollständig abgearbeitet, insbesondere Snapshot der Ziel-VM. **Durchführung:** Richte auf der Ziel-VM eine Dauerbeobachtung der SYN_RECV-Verbindungen ein (`ss -tan state syn-recv`, wiederholt ausgeführt, siehe Kapitel 8.2). Führe dann **ausschließlich den ratenbegrenzten Test aus Beispiel 8.1b** aus (nicht `--flood`). **Erwartetes Ergebnis:** Ein kurzzeitiger, messbarer Anstieg der SYN_RECV-Einträge während der rund zwei Sekunden des Tests, danach Rückgang auf den Ausgangswert. **Auswertung:** Dokumentiere Ausgangswert, Spitzenwert und Zeit bis zur Erholung – diese drei Zahlen sind die Grundlage für den Vergleich in Übung 8. **Fehlerbehebung:** Siehst du gar keinen Anstieg, prüfe, ob Port 80 auf der Ziel-VM tatsächlich einen Dienst betreibt; ein sofort mit RST beantworteter, geschlossener Port reserviert keine SYN_RECV-Ressource.

Übung 8: Gegenmaßnahme vergleichen

Ziel: Die Wirkung von SYN Cookies auf dieselbe Testlast direkt vergleichen. **Voraussetzungen:** Ergebnis aus Übung 7 liegt vor; Root-Zugriff auf die Ziel-VM. **Durchführung:** Aktiviere SYN Cookies auf der Ziel-VM (`sysctl -w net.ipv4.tcp_syncookies=1`) und wiederhole Übung 7 identisch, einschließlich desselben ratenbegrenzten Befehls aus Beispiel 8.1b. **Erwartetes Ergebnis:** Ein deutlich geringerer oder stabilerer Anstieg der SYN_RECV-Einträge als in Übung 7. **Auswertung:** Der Unterschied zwischen beiden Messreihen ist der praktische Nachweis dessen, was Kapitel 8.3 als Gegenmaßnahme beschreibt. **Fehlerbehebung:** Zeigt sich kein Unterschied, prüfe mit `sysctl net.ipv4.tcp_syncookies`, ob die Einstellung tatsächlich auf `1` steht, und ob dein Kernel SYN Cookies überhaupt unterstützt.

> **WARNUNG – Sicherheitsrisiko**
>
> Übung 7 und 8 verwenden bewusst **nicht** `--flood`, sondern den ratenbegrenzten Befehl aus Kapitel 8.1.1. Wer zusätzlich die ungebremste Variante aus Kapitel 8.1.2 beobachten möchte, wiederholt beide Übungen ein weiteres Mal, hält die Laufzeit dabei aber auf wenige Sekunden begrenzt und beachtet die Warnhinweise in Kapitel 8 erneut.

Zusammenfassung

Zusammenfassung

- Dasselbe Internal-Network-Labor wie im Ethical-Hacking-Handbuch reicht für alle acht Übungen aus.
- Jede Übung folgt demselben Schema aus Ziel, Voraussetzungen, Durchführung, erwartetem Ergebnis, Auswertung und Fehlerbehebung.
- Abweichende Ergebnisse sind keine Fehler per se, sondern hängen oft von Betriebssystem, Kernel und Firewall-Konfiguration ab.
- Übung 7 und 8 nutzen bewusst den ratenbegrenzten Test, nicht die ungebremste `--flood`-Variante.

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen

Lernziele

- Du kannst die häufigsten hping3-Fehlermeldungen einordnen und beheben.
- Du hast einen kompakten Spickzettel für den Alltag zur Hand.
- Du kennst weiterführende Ressourcen für die Vertiefung.

11.1 Häufige Probleme

Meldung / Symptom	Wahrscheinliche Ursache	Lösung
`socket(): Operation not permitted`	fehlende Rechte für Rohsockets	`sudo` nutzen oder `cap_net_raw`-Capability setzen (Beispiel 3.2)
Dauerhaft keine Antwort, obwohl Port sicher offen ist	falsches Netzwerkinterface oder fehlerhafte Testnetz-Isolation	`ip route`/`ip a` prüfen, richtiges Interface mit `-I` angeben
`--flood` reagiert gar nicht sichtbar	Zielsystem oder Netzwerk puffert/verwirft bereits auf niedrigerer Ebene	Messgrößen aus Kapitel 8.2 direkt auf dem Zielsystem prüfen, nicht nur die hping3-Ausgabe
Scan-Ergebnisse weichen von Nmap ab	unterschiedliche Timeout- oder Wiederholungseinstellungen	Wiederholungen erhöhen (`-c`), Ergebnis mehrfach prüfen
`hping3` öffnet keine `hping3>`-Eingabeaufforderung	ohne Tcl-Unterstützung kompiliert/installiert	Distributionspaket mit Tcl-Unterstützung installieren oder aus den Quellen bauen

11.2 Spickzettel

Beispiel 11.1: Die wichtigsten hping3-Befehle auf einen Blick

```
# Einzelnen Port prüfen
sudo hping3 -S -p <port> -c 1 <ziel>

# Mehrere Ports (eingebauter Scan)
sudo hping3 --scan <start>-<ende> -S <ziel>

# UDP-Port prüfen
sudo hping3 -2 -p <port> -c 1 <ziel>
# TTL und Fenstergröße einer Antwort beobachten
sudo hping3 -S -p <port> -c 1 -V <ziel>

# Fragmentierung erzwingen (Firewall-Test)
sudo hping3 -S -p <port> -c 1 -f <ziel>

# Quellport setzen (Firewall-Regel-Test)
sudo hping3 -S -p <port> -s <quellport> -c 1 <ziel>

# Interaktive Tcl-Umgebung starten
hping3

# Ein Tcl-Skript ausführen
hping3 exec skript.htcl
```

11.3 Glossar

- **Fenstergröße (Window Size)** – gibt an, wie viele Bytes ungestützt empfangen werden können; dient als Hinweis beim Betriebssystem-Fingerprinting.
- **Idle Scan** – ein Portscan, der die eigene Quelladresse verschleiert, indem die IP-ID-Sequenz eines dritten „Zombie“-Hosts ausgewertet wird.
- **IP-ID** – ein Zählerfeld im IP-Header, dessen Verlauf beim Idle Scan ausgewertet wird.
- **Paketgenerator** – ein Werkzeug, das einzelne Netzwerkpakete manuell und feldgenau zusammenstellt, statt feste Abläufe zu automatisieren.
- **SYN Cookie** – eine Serverseitige Gegenmaßnahme gegen SYN-Floods, die den Verbindungszustand in die Sequenznummer kodiert, statt Ressourcen vorab zu reservieren.
- **SYN-Flood** – eine Technik, die Server-Ressourcen durch massenhafte, nie abgeschlossene Verbindungsanfragen erschöpft.
- **TTL (Time to Live)** – die Anzahl verbleibender Netzwerk-Hops, bevor ein Paket verworfen wird; dient ebenfalls als Fingerprinting-Hinweis.
- **Zombie (Idle Scan)** – ein dritter Host mit vorhersagbarer IP-ID-Sequenz, der für einen Idle Scan als „stiller Vermittler“ genutzt wird.

11.4 Weiterführende Ressourcen

Ressource	Inhalt
Offizielle hping3-Manpage (`man hping3`)	vollständige, verbindliche Optionsreferenz
Ethical-Hacking-Handbuch (dieselbe Buchreihe)	Gesamtzusammenhang: Recon, Scanning, Exploitation, Reporting
RFC 793 (TCP) und RFC 791 (IP)	die zugrunde liegenden Protokollstandards
TryHackMe / HackTheBox	legale Übungsplattformen, auf denen sich Portscanning und Firewall-Konzepte vertiefen lassen
Wireshark	ergänzendes Werkzeug, um die mit hping3 gesendeten Pakete direkt mitzuschneiden und zu verifizieren

TIPP – Profi-Trick

Der schnellste Weg, hping3 wirklich zu verstehen, ist die Kombination mit **Wireshark**: Starte einen Mitschnitt auf der Kali-VM, sende ein Paket mit hping3, und sieh dir danach im Wireshark-Fenster jedes Feld an, das du gerade erzeugt hast.

Zusammenfassung

Zusammenfassung

- Die häufigsten Probleme betreffen fehlende Rechte, falsche Interfaces oder eine unvollständig isolierte Testumgebung.
- Der Spickzettel in diesem Kapitel deckt die acht häufigsten Befehlsmuster des gesamten Buches ab.

- Die offizielle Manpage, das Ethical-Hacking-Handbuch, die zugrunde liegenden RFCs und Wireshark sind die wichtigsten Anschlussstellen zum Weiterlernen.

Verzeichnis der Befehlsbeispiele

Beispiel 2.1: Der TCP-Drei-Wege-Handshake in hping3-Begriffen … Seite 9
Beispiel 3.1: hping3 installieren … Seite 10
Beispiel 3.2: hping3 mit den nötigen Rechten ausführen … Seite 10
Beispiel 3.2b: Capability statt dauerhafter root-Rechte … Seite 11
Beispiel 3.3: Minimales Testnetz für dieses Buch … Seite 11
Beispiel 3.4: Isolation vor jeder Übung verifizieren … Seite 11
Beispiel 4.1: Grundstruktur … Seite 12
Beispiel 4.2: Ein einzelnes SYN-Paket senden und die Antwort lesen … Seite 13
Beispiel 4.3: Nützliche Zusatzoptionen … Seite 13
Beispiel 5.1: Einen einzelnen Port testen … Seite 14
Beispiel 5.2: Mehrere Ports nacheinander prüfen … Seite 14
Beispiel 5.3: Portbereich mit --scan … Seite 14
Beispiel 5.4: Einen UDP-Port testen … Seite 15
Beispiel 6.1: Denselben Port mit unterschiedlichen Flags testen … Seite 16
Beispiel 6.2: Fragmentierung erzwingen … Seite 16
Beispiel 6.3: Mit einem vertrauenswürdig wirkenden Quellport testen … Seite 16
Beispiel 6.4: Ein bekanntes Testmuster senden und im IDS-Log prüfen … Seite 17
Beispiel 7.1: Idle Scan in drei Schritten (Konzept) … Seite 18
Beispiel 7.2: Idle Scan mit hping3 vorbereiten (IP-ID eines Zombies beobachten) … Seite 18
Beispiel 7.3: TTL einer Antwort auswerten … Seite 19
Beispiel 8.1: Das Grundmuster eines SYN-Floods (Konzept) … Seite 20
Beispiel 8.1b: Ratenbegrenzter SYN-Test – ausschließlich gegen die eigene Testumgebung … Seite 20
Beispiel 8.1c: Ungebremster SYN-Flood – ausschließlich gegen die eigene Testumgebung, mit besonderer Vorsicht … Seite 21
Beispiel 9.1: Interaktiven Tcl-Modus starten … Seite 23
Beispiel 9.1b: Ein Tcl-Skript ausführen … Seite 23
Beispiel 9.2: hping-Send/Receive-Zyklus in Tcl (vereinfachtes Demonstrationsbeispiel) … Seite 23
Checkliste 10.1: Vor jeder Übung … Seite 25
Beispiel 11.1: Die wichtigsten hping3-Befehle auf einen Blick … Seite 29

Index

--flood – 21, 27
--scan – 14
ACK – 4, 9, 11, 13, 20, 22, 25, 26
Capability – 11
Fenstergröße – 6, 9, 19, 30
FIN – 4, 6, 9, 13, 16, 19, 26, 27
Firewall-Test – 9
Fragmentierung – 6, 16, 27
hping3 – 4, 6, 7, 8, 9, 10, 13, 14
Idle Scan – 4, 6, 18, 30
IDS – 4, 6, 17
IP-ID – 18, 27, 30
Paketgenerator – 6, 30
Portscanning – 4, 6
RST – 4, 6, 13, 14, 16, 21, 23, 26
SYN – 13, 16, 20, 21, 26, 27, 30
SYN Cookie – 21, 27, 30
SYN-Flood – 20, 21, 30
Tcl – 23, 24
TCP-Flags – 26
TTL – 6, 19, 26, 30
UDP – 4, 6, 15
Zombie – 6, 18, 27, 30

HPING3 DAS PRAXISHANDBUCH

Was ein einzelnes Paket über ein Netzwerk verrät

hping3 erzeugt TCP-, UDP-, ICMP- und Rohpakete von Hand – Feld für Feld. Statt feste Abläufe zu automatisieren wie Nmap, gibt dir dieses Werkzeug die volle Kontrolle über Flags, TTL, Fenstergröße und mehr. Dieses Handbuch führt vom TCP/IP-Fundament über Portscanning und Firewall-Tests bis zu Idle Scan, OS-Fingerprinting und den Grundlagen von SYN-Floods – durchgehend mit klarer Trennung zwischen Testlabor und realem Netz.

DAS ERWARTET DICH

- TCP/IP-Grundlagen: Header-Felder, Flags und der Drei-Wege-Handshake im Detail
- Portscanning, Firewall- und IDS-Tests mit gezielt konstruierten Paketen
- Idle Scan (Zombie-Scan) und Betriebssystem-Fingerprinting über TTL und Fenstergröße
- SYN-Flood-Grundlagen und Gegenmaßnahmen (SYN Cookies, Rate Limiting) – nur im eigenen Labor
- 8 geführte Übungen, Spickzettel, Glossar und Troubleshooting

Für wen?

Leser:innen des Ethical-Hacking-Handbuchs, die ein einzelnes Werkzeug vertiefen möchten, sowie Studierende und Auszubildende, die TCP/IP auf Feldebene verstehen wollen. Ausschließlich für legale, autorisierte Tests und eigene Laborumgebungen.

33 Seiten · Stand Oktober 2026 · Ergänzung zum Ethical-Hacking-Handbuch

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.